

Seminar Report

Dynamic and Adaptive Attention in Modern Transformers

Nirish Samant

MatrNr: 18460226

Supervisor: Lauritz Rasbach

Georg-August-Universität Göttingen
Institute of Computer Science

September 30, 2026

Abstract

Attention has become a fundamental mechanism in modern neural network architectures. This report examines the development of attention from its role in addressing the limitations of recurrent neural networks and sequence-to-sequence models to its central role in Transformer architectures. It first introduces self-attention and multi-head attention as the primary mechanisms used by Transformers to model dependencies between tokens. The report then explores the computational and memory challenges associated with attention, particularly the quadratic complexity of full attention and the growing cost of the key-value cache during long-context inference. Several techniques for improving efficiency are discussed, including Grouped-Query Attention, Multi-Query Attention, PagedAttention, FlashAttention, quantization, sparse attention, and linear attention. Finally, the report explores advanced attention mechanisms, including dynamic and learned sparse attention, adaptive attention spans, and dynamic token selection and merging. Overall, the report demonstrates how attention has evolved from a solution to the limitations of recurrent architectures into a flexible framework in which computation can be dynamically allocated according to the input, available resources, and modeling requirements.

Keywords — Attention Mechanisms, Transformer Architecture, Self-Attention, Multi-Head Attention, Long-Context Modeling, KV Cache, Attention Efficiency, FlashAttention, Grouped-Query Attention, Multi-Query Attention, Sparse Attention, Dynamic Attention, Adaptive Attention, Token Pruning, Token Merging

AI Usage Transparency Statement

- **How was AI used in the ideation process?** *Response: AI tools were used as a supporting aid during the development and organization of ideas for the report. They helped refine the focus of the topic, structure the discussion, and identify possible aspects of attention mechanisms that could be covered. The final selection and organization of the topics were made by me.*
- **How was AI used in working with literature and sources?** *Response: AI tools were used to support the understanding and organization of technical literature and concepts related to attention mechanisms and Transformer architectures. AI was not treated as a primary academic source. The relevant academic references and cited sources were selected and used by me, and the technical content was reviewed in the context of the original sources.*
- **How was AI used in the process of academic writing?** *Response: AI tools were used as a writing-support tool to improve the clarity, structure, grammar, and wording of parts of the report. They were also used to help refine the abstract, keywords, and conclusion. I remain responsible for the content, organization, technical accuracy, and final version of the report.*
- **How was AI used in methodology, particularly in data collection and analysis?** *Response: The report is primarily a literature-based review and does not involve the collection or statistical analysis of an original dataset. AI tools were therefore not used to collect or analyze experimental data. AI was used only as a supporting tool for organizing and understanding the concepts discussed in the literature.*
- **How was AI used in other activities?** *Response: AI tools were additionally used for language refinement, proofreading, and improving the presentation of written material. They were also used to assist with writing code and to support practical engagement with the topic. They were used as an assistance tool rather than as a replacement for researching and presenting the material.*

I hereby declare that all uses of AI tools in this work have been fully and accurately disclosed. Missing or incorrect information may be considered an attempt to cheat.

Contents

List of Tables	v
List of Figures	v
List of Abbreviations	vi
1 Introduction and Motivation	1
1.1 Limitations of Recurrent Architectures	1
1.2 The Bottleneck in Sequence-to-Sequence Models	1
1.3 Attention as a Solution	1
2 Attention in the Transformer Architecture	2
2.1 Self-Attention	2
2.2 Multi-Head Attention	3
3 Modern Transformer Architectures and Efficiency	3
3.1 From Full Attention to Resource Allocation	4
3.2 The KV Cache and the Memory Wall	4
3.3 Prefill vs. Decode: Diverging Bottlenecks	5
3.4 Grouped-Query and Multi-Query Attention	5
3.5 PagedAttention	6
3.6 FlashAttention	6
3.7 Model Quantization	7
3.8 Sparse Attention	7
3.8.1 Longformer	8
3.8.2 BigBird	9
3.9 Linear Attention	9
3.9.1 Performer	10
3.9.2 Linformer	10
3.10 Comparison of Approaches	10
4 Learned Sparse Attention	11
4.1 Learned and Trainable Dynamic Sparse Attention	11
4.2 Dynamic Attention Masks and Content-Aware Sparsity	12
4.3 Top-k Attention and Dynamic Sparsity	12
5 Adaptive Attention Span	13
5.1 Learnable Attention Span	13
5.2 Input-Dependent Context Length	13
5.3 Dynamic Masking and Adaptive Context Windows	14
6 Dynamic Token Selection, Pruning, and Merging	14
6.1 Adaptive and Importance-Based Token Selection	14
6.2 Token Pruning and Progressive / Dynamic Eviction	14
6.3 Token Merging and Candidate Token Selection	14

7 Conclusion	15
References	16
A Transformer Architecture and Model Representation	A1
A.1 Attention in the Encoder and Decoder	A1
A.2 Positional Information: Rotary Positional Embeddings	A1
A.3 Quantization	A1
B Sparse Attention Benchmark Comparisons	A2
B.1 Longformer	A3
B.2 BigBird	A3
C Dynamic Attention Mechanisms	A4
C.1 Query-Dependent Attention	A4
C.2 Context-Dependent and Content-Aware Attention	A4
C.3 Dynamic and Importance-Based Attention Allocation	A5
D Attention Interpretability	A5
D.1 Attention as a Graph	A5
D.2 Attention Rollout	A6
D.3 Attention Flow	A6

List of Tables

1	Comparison of modern attention-efficiency techniques.	10
2	Average accuracy (%) on the Long Range Arena benchmark [Tay+21]. All models share the same backbone and training setup; only the attention pattern differs.	A2
3	Longformer-base vs. RoBERTa-base on long-document QA (development sets) [BPC20].	A3
4	QA development-set results, base-size models: RoBERTa vs. Longformer vs. BigBird [Zah+20].	A3

List of Figures

1	Encoder–Decoder RNN architecture illustrating the information bottleneck.	2
2	Visualization of attention patterns across different heads in the same Transformer layer.	4
3	Key Value sharing in Multi-Head Attention (MHA), Grouped-Query Attention (GQA), and Multi-Query Attention (MQA). Fewer KV heads mean a smaller KV cache.	6
4	Comparison of full, causal, and local sliding-window attention patterns. . .	8
5	Illustration of different sparse attention mechanisms. (Illustrative, with random scores for top- k and random links.)	9
6	Memory of a 70B-parameter model at different precisions.	A2

List of Abbreviations

AI	Artificial Intelligence
BERT	Bidirectional Encoder Representations from Transformers
BF16	Brain Floating Point (16-bit)
CLS	Classification token
CNN	Convolutional Neural Network
DAG	Directed Acyclic Graph
FP16	16-bit Floating Point
FP32	32-bit Floating Point
GPU	Graphics Processing Unit
GQA	Grouped-Query Attention
HBM	High Bandwidth Memory
INT4	4-bit Integer
KV	Key-Value
LLM	Large Language Model
LSTM	Long Short-Term Memory
MHA	Multi-Head Attention
MQA	Multi-Query Attention
NLP	Natural Language Processing
RNN	Recurrent Neural Network
RoPE	Rotary Position Embedding
Seq2Seq	Sequence-to-Sequence
SOTA	State of the Art
SRAM	Static Random-Access Memory
ToMe	Token Merging
TPU	Tensor Processing Unit
ViT	Vision Transformer
vLLM	Virtual Large Language Model inference engine

1 Introduction and Motivation

Natural language is inherently sequential, and many of the phenomena that make it difficult to model computationally like word alignment, coreference, and long-distance syntactic agreement are **long-range** in nature. A pronoun such as “it” may refer to a noun mentioned many sentences earlier, and a correct translation of a sentence may require reordering words that are far apart in the source language [Uni23]. Before the introduction of attention, the dominant architectures for such tasks were **RNN** and their gated variant, the **LSTM**.

1.1 Limitations of Recurrent Architectures

RNNs process a sequence one token at a time, propagating information through a chain of hidden states $h^{(1)}, h^{(2)}, \dots, h^{(T)}$. This design has two structural weaknesses. First, because each hidden state depends on the previous one, computation **cannot be parallelized** across the sequence, which severely limits training throughput on modern hardware. Second, information about early tokens must survive many multiplicative updates to influence later predictions, which causes the well-known **vanishing gradient** problem. The influence of distant tokens decays rapidly, and the network is biased toward the most recent states.

LSTMs mitigate the vanishing gradient problem through **gated memory cells** that allow gradients to flow more easily across time steps [Ola15]. However, LSTMs do not solve the underlying scalability problem: computation remains strictly sequential, and a fixed-size hidden state must still summarize an arbitrarily long history, so the “**memory**” **bottleneck** persists [Ola15]. Here, “memory” refers to the LSTM’s hidden and cell states, which carry a fixed-size representation of the sequence seen so far.

1.2 The Bottleneck in Sequence-to-Sequence Models

The Seq2Seq architecture applies an encoder RNN to compress a source sentence into a single fixed-size vector, which is then used to initialize a decoder RNN that generates the target sentence autoregressively [BCB15]. This design creates an **information bottleneck**: the final encoder hidden state must capture everything relevant about the source sentence (see Fig. 1), regardless of its length, before the decoder has produced a single output token [BCB15]. For long sentences this compression is lossy, and translation quality degrades noticeably as sentence length increases.

1.3 Attention as a Solution

Attention was introduced to let the decoder look directly at every encoder hidden state at every decoding step, rather than relying solely on a single compressed summary [BCB15]. At each decoding step t , the decoder computes a set of **attention scores** between its current hidden state s and each encoder hidden state h_i , typically via a dot product:

$$e = [s^T h_1, \dots, s^T h_N] \in \mathbb{R}^N. \quad (1)$$

A softmax normalizes these scores into an **attention distribution** $\alpha = \text{softmax}(e)$, which is then used to compute a weighted sum of the encoder states, called the **attention**

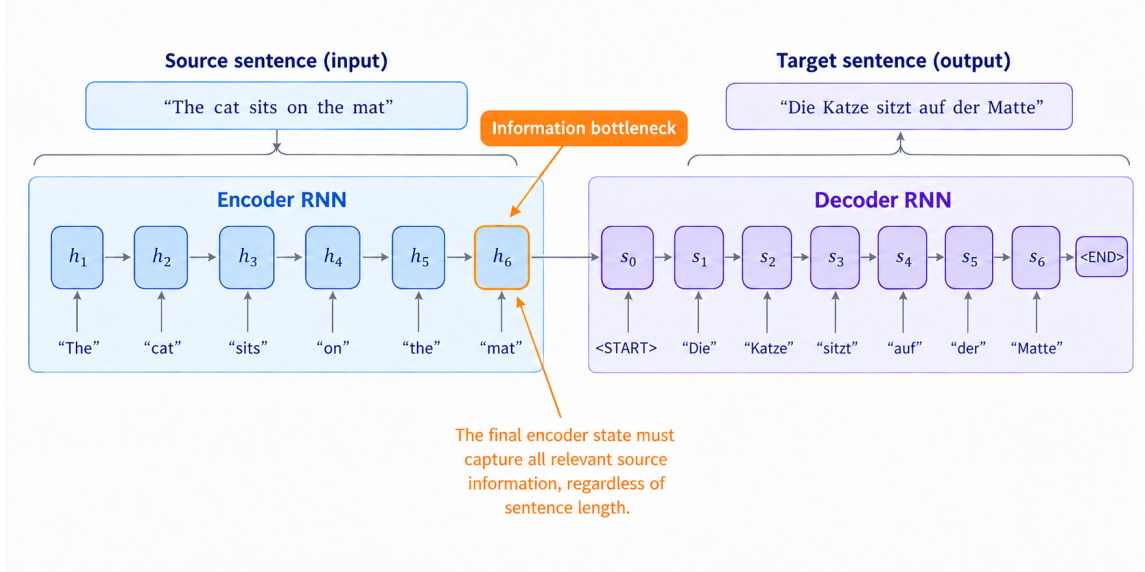


Figure 1: Encoder–Decoder RNN architecture illustrating the information bottleneck.

output or context vector:

$$a = \sum_{i=1}^N \alpha_i h_i. \quad (2)$$

This mechanism directly addresses the two central weaknesses of recurrent models. First, it bypasses the information bottleneck by giving the decoder direct access to all source states rather than a single fixed-size summary [BCB15]. Second, it provides a shortcut for gradients to flow to distant encoder states, alleviating the vanishing gradient problem [BCB15]. Beyond these architectural benefits, attention also has the practical advantage of **interpretability**: the distribution α indicates, at least approximately, which input tokens the model considered relevant when producing a given output token [WP19]. A detailed discussion of attention interpretability, including attention rollout and attention flow, is provided in Appendix D.

2 Attention in the Transformer Architecture

The Transformer, introduced by Vaswani et al. [Vas+17], is a neural network architecture centered on **attention** mechanisms. It consists of an encoder, which builds a contextual representation of the input, and a decoder, which generates output tokens autoregressively while attending to the encoder’s representations. Both are built from a small set of reusable components: multi-head self-attention, position-wise feed-forward networks, and residual connections combined with layer normalization [Vas+17].

2.1 Self-Attention

Self-attention is the mechanism that allows every token in a sequence to attend to every other token in the same sequence. Concretely, each input embedding x_i is projected into

three separate vectors using learned weight matrices:

$$q_i = x_i W^Q, \quad k_i = x_i W^K, \quad v_i = x_i W^V, \quad (3)$$

referred to as the **query**, **key**, and **value**, respectively. The matrices W^Q and W^K both have dimensions $d \times d_k$, while W^V has dimensions $d \times d_v$; d , d_k , and d_v are hyperparameters, with d_k and d_v typically much smaller than the model dimension d . All three matrices are randomly initialized and learned during training.

Given the matrices Q , K , and V stacking the queries, keys, and values of all tokens, the output of self-attention is a weighted sum of the value vectors, where the weight assigned to each value is computed from the inner product of the corresponding query and key, scaled by $\sqrt{d_k}$ and normalized with a softmax:

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (4)$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large in magnitude as d_k increases, which would otherwise push the softmax into regions with extremely small gradients [Vas+17].

2.2 Multi-Head Attention

A single self-attention computation forces the model to average over all types of relevant relationships between tokens using one shared set of query, key, and value projections. Multi-Head Attention (MHA) instead performs attention multiple times in parallel, using separate learned projections for each head. Each head can then specialize in a different aspect of language, such as local semantics, grammatical agreement, coreference resolution, or semantic role assignment. As a concrete example, the base configuration of **BERT** computes **twelve** attention heads per self-attention layer.

Formally, the embedding space of dimension d_{model} is split evenly across h heads, so that each head operates on a reduced dimensionality $d_k = d_{\text{model}}/h$. Each head computes its own attention output,

$$\text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right), \quad (5)$$

after which the outputs of all heads are concatenated and projected back to the model dimension using a learned output matrix W^O :

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O. \quad (6)$$

Visualizations of trained Transformers confirm that different heads do indeed attend differently to the same input (see Fig. 2). For example, distinct heads in the same layer have been shown to specialize in resolving pronoun coreference versus attending to local syntactic neighbors [Ala18].

3 Modern Transformer Architectures and Efficiency

The original Transformer formulation is elegant but computationally demanding: self-attention has **quadratic time and memory complexity** in the sequence length, and

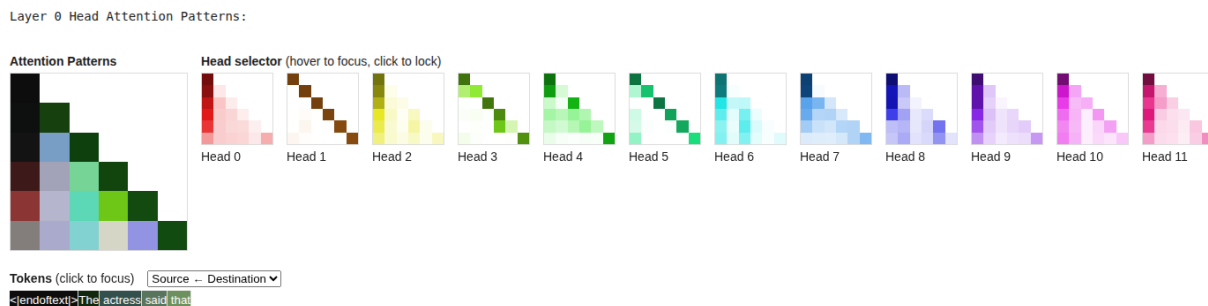


Figure 2: Visualization of attention patterns across different heads in the same Transformer layer.

generating text autoregressively requires sequential decoding, with each new token attending to the growing cached context. As models and context windows have scaled by orders of magnitude, a large body of work has focused not on increasing the expressiveness of attention, but on making it **dynamic, adaptive, and efficient** enough to deploy at scale. This section surveys the main techniques. A more detailed discussion of dynamic attention mechanisms is provided in Appendix C.

3.1 From Full Attention to Resource Allocation

The original Transformer applies dense attention: every position can attend to every other permitted position. This is expressive, but it does not distinguish between interactions that are essential and interactions that contribute little to the current computation.

This motivates the broader notion of **adaptive attention**. An adaptive mechanism can change the amount, location, representation, or precision of computation according to the input, model state, or inference phase. The adaptation may happen at several levels:

- **Interaction level:** restrict which tokens attend to which other tokens.
- **Head level:** share or reduce key-value representations across attention heads.
- **Memory level:** store and retrieve cached states more efficiently.
- **Numerical level:** reduce the precision used to represent model states.
- **Kernel level:** reorganize computation to reduce data movement.

These mechanisms should not be treated as interchangeable. Some preserve the exact mathematical attention result while changing its implementation, whereas others deliberately modify or approximate the interaction pattern.

3.2 The KV Cache and the Memory Wall

During autoregressive decoding, the model generates one token at a time; naively, this would require re-running the entire decoder stack over the whole generated sequence for every new token, which is prohibitively expensive. The KV cache stores the key and value vectors computed for all previous tokens, so that decoding a new token only requires

computing its own query, key, and value and attending to the cached history [Wel25]. This trades increased memory usage for a substantial reduction in redundant computation.

The size of this cache, however, grows linearly with the number of layers, attention heads, head dimension, and sequence length. For a model with l layers, n_h key/value heads, head dimension d_h , storing keys and values (factor of 2) at 2 bytes per number (FP16), the number of bytes required for a context of n tokens is

$$\text{Cache size} = 2 \cdot n \cdot d_h \cdot n_h \cdot l \cdot (\text{bytes per number}). \quad (7)$$

For a conventional MHA-style hypothetical DeepSeek R1/V3-scale configuration ($d_h = 128$, $n_h = 128$, $l = 61$), this amounts to roughly 4 MB per token, so a context of 100,000 tokens requires on the order of 400 GB of cache memory, far exceeding what a single accelerator can hold [Wel25].

This growing cost is a symptom of a broader trend known as the **memory wall**. It is the gap between the rapid growth of raw compute throughput on GPU and TPU and the much slower growth of memory bandwidth.

3.3 Prefill vs. Decode: Diverging Bottlenecks

During prefill, the full prompt of length n is processed in a single forward pass: Q , K , and V are computed for all n positions at once, and the attention score matrix $QK^\top$ is a dense $n \times n$ matrix multiplication. Because the same weight matrices are reused across all n tokens simultaneously, this operation has high **arithmetic intensity** i.e. the GPU spends most of its time doing actual multiply-accumulate work rather than waiting on memory transfers [Pop+23; Wel25]. Prefill is therefore **compute-bound**, and its cost scales as $O(n^2d)$ **FLOPs** for full attention.

During autoregressive decoding, exactly one new token is generated per step, so each step’s attention computation reduces to a single query vector against the cached keys, $q_t k_{1:t}^\top$ i.e. a vector-matrix product rather than a matrix-matrix product. This has very low arithmetic intensity: only a handful of FLOPs are performed for every byte of the KV cache [Pop+23]. Decode is therefore **memory-bound**: as discussed in Section 3.2, the KV cache grows linearly with the number of tokens already generated, and re-reading this ever-larger cache from memory at every step is what dominates decode latency and puts sustained pressure on VRAM capacity and memory bandwidth.

As a result, LLM inference is frequently **memory-bound rather than compute-bound**, particularly during sequential decoding, where model weights and the **KV cache must be repeatedly streamed from memory** for every single generated token. The subsequent subsections describe techniques like Grouped-Query Attention (GQA), Multi-Query Attention (MQA), PagedAttention, FlashAttention, and quantization that are all, in one way or another, responses to this memory wall.

3.4 Grouped-Query and Multi-Query Attention

In standard multi-head attention, every query head has its own dedicated key and value projections, $\text{head}_i \rightarrow (Q_i, K_i, V_i)$. Since the KV cache must store keys and values for every head, this leads to a KV cache that scales linearly with the number of heads, which becomes the dominant memory cost for long sequences [Ain+23].

GQA addresses this by partitioning the query heads into **groups**, where all query heads within a group share a single set of key and value projections [Ain+23]. This

reduces the KV cache size roughly in proportion to the number of groups, at the cost of a small reduction in the model’s representational capacity, since heads within a group can no longer attend to independently computed keys and values.

MQA takes this idea to its extreme: all query heads share a **single key and value head** [Ain+23]. This minimizes the KV cache size and maximizes decoding speed, but comes with a larger trade-off in model quality than GQA, since all heads are now forced to share exactly the same keys and values (see Fig. 3). GQA is generally considered as a middle ground that recovers most of the speed benefit of MQA while retaining most of the expressiveness of full multi-head attention [Ain+23].

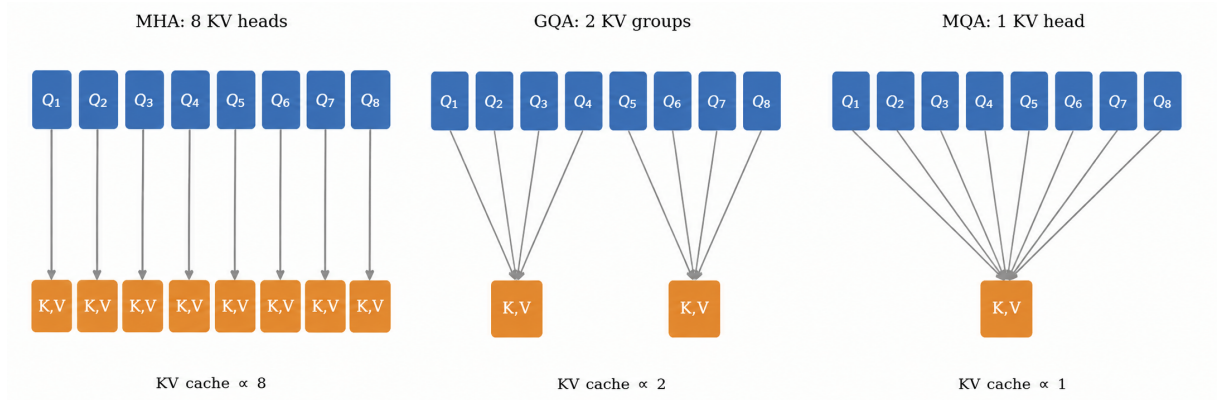


Figure 3: Key Value sharing in Multi-Head Attention (MHA), Grouped-Query Attention (GQA), and Multi-Query Attention (MQA). Fewer KV heads mean a smaller KV cache.

3.5 PagedAttention

Even with a reduced number of KV heads, serving many **concurrent requests** efficiently requires careful memory management. Conventional implementations allocate the KV cache for each sequence as one large contiguous block of memory, which leads to memory fragmentation, wasted GPU memory, and reduced serving throughput, especially since the final length of a generated sequence is not known in advance [Kwo+23].

PagedAttention addresses this by treating the KV cache analogously to virtual memory paging in operating systems: the cache is divided into small, fixed-size blocks (“**pages**”), and logical token positions are mapped to physical memory pages that can be allocated, shared, and reused dynamically as needed [Kwo+23]. This design, used in the vLLM serving system, substantially improves GPU memory utilization, reduces fragmentation, and increases overall serving throughput for LLM inference [Kwo+23].

3.6 FlashAttention

Standard implementations of attention explicitly materialize the full $n \times n$ attention score matrix QK^T in memory before applying the softmax, which requires $O(n^2)$ memory and involves repeatedly writing and reading this large matrix from slow off-chip memory [Dao+22]. FlashAttention avoids ever materializing the full matrix by computing attention in small tiled **blocks**, keeping intermediate results in the **GPU’s fast on-chip SRAM** rather than its much slower High Bandwidth Memory (HBM) [Dao+22].

FlashAttention splits K and V into blocks of B_c rows and Q into blocks of B_r rows, chosen so that the blocks fit into SRAM [Dao+22]. Two nested loops are used: the outer loop loads one block K_j, V_j from HBM into SRAM. The inner loop goes over all blocks Q_i , computes the scores $S_{ij} = Q_i K_j^T$ in SRAM, updates the softmax statistics and the running output O_i , and writes O_i back to HBM. The full $n \times n$ matrix is never stored; only small blocks of size $B_r \times B_c$ exist at any time, and only in SRAM.

Softmax needs the sum over a whole row of scores, but FlashAttention only sees one block of the row at a time. The softmax trick here is to keep just two numbers per row, the running maximum m and the running sum of exponentials ℓ , and to *rescale* them whenever a new block arrives [Dao+22]. If the new block contains a larger maximum m_{new} , the old sum and the old partial output are multiplied by $e^{m_{\text{old}} - m_{\text{new}}}$, which shifts them to the new reference value. Then the contribution of the new block is added:

$$\ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} + \ell_{\text{block}}. \quad (8)$$

The maximum is subtracted anyway to avoid overflow, and it cancels out in the final division. After the last block, the result is exactly the same as with the full softmax, so FlashAttention is exact and not an approximation [Dao+22].

By **fusing** the matrix multiplication, masking, softmax, and output computation into a single kernel that operates block-by-block, FlashAttention drastically reduces the number of memory reads and writes, thus substantially reducing memory traffic and avoiding the $O(n^2)$ intermediate attention matrix, while computing the mathematically **exact** same attention output as the naive implementation [Dao+22]. This yields substantial wall-clock speedups without any loss of accuracy, unlike approximate methods such as sparse or linear attention.

3.7 Model Quantization

Model quantization **reduces the numerical precision** used to store model parameters (and, in some settings, the KV cache), directly lowering memory usage and the associated memory bandwidth required to stream weights during inference. Since the memory required to store a model scales as

$$\text{Memory} = \text{Number of parameters} \times \text{bits per parameter}, \quad (9)$$

switching from 32-bit floating point (FP32) to lower-precision formats such as FP16/BF16, INT8, or INT4 can reduce memory requirements by up to a factor of eight (see Appendix A.3). The benefits of quantization are therefore lower memory bandwidth requirements, a smaller overall model footprint, and faster inference, at the potential cost of some accuracy degradation if precision is reduced too aggressively.

3.8 Sparse Attention

Full self-attention permits every token to interact with every other token, giving quadratic pairwise interaction complexity. Sparse attention restricts each token to attend only to a **subset of other tokens**, instead of the full sequence.

Common patterns include **local attention**, where tokens attend only within a sliding window of nearby tokens (see Fig. 4); **global attention**, where a small number of designated tokens attend to (and are attended to by) the entire sequence; and **random**

attention, which adds sparse random long-range connections. Combining these patterns reduces complexity to $O(n)$ or $O(n \log n)$ while still enabling reasonably effective long-context modeling. Models such as Longformer and BigBird demonstrate different approaches to sparse interaction patterns. For a detailed benchmark comparison of sparse attention mechanisms, see Appendix B.

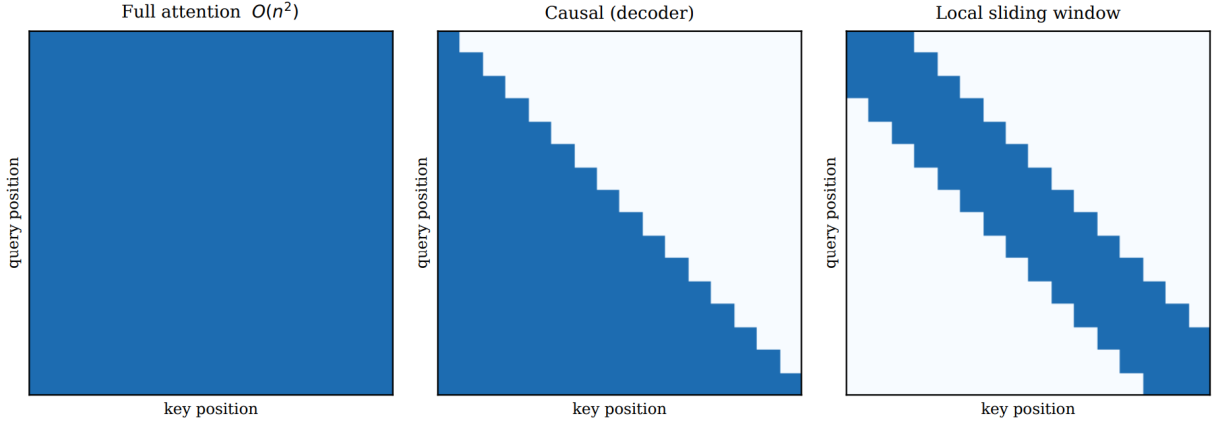


Figure 4: Comparison of full, causal, and local sliding-window attention patterns.

The advantage is reduced computation and memory growth. The cost is that an omitted interaction cannot contribute directly at that layer. A sparse design therefore makes an explicit assumption about which interactions are likely to matter. This is a direct example of adaptive attention as **selective connectivity**.

3.8.1 Longformer

Longformer combines a fixed-size sliding-window local attention with a small number of task-specific global attention tokens (e.g. [CLS]), which attend to and are attended to by the entire sequence [BPC20]. This hybrid pattern keeps the per-layer cost linear in sequence length while still allowing selected tokens to aggregate information globally, making it well suited to long-document tasks such as question answering and summarization. Benchmark results for Longformer are provided in Appendix B.1.

The sliding-window attention itself is implemented with a fixed window size w , so each token attends only to $w/2$ neighboring tokens on either side, giving a per-layer complexity of $O(n \times w)$ rather than the $O(n^2)$ of full self-attention [BPC20]. To increase the receptive field without added cost, Longformer also introduces **dilated** sliding windows in some heads, analogous to dilated convolutions, so that different attention heads can specialize in modeling either local syntactic context or longer-range dependencies [BPC20].

The global attention tokens are task-specific: for classification, the [CLS] token is given global attention, whereas for QA, all question tokens are treated as global so that they can directly attend to and be attended to by the entire document [BPC20]. Because the global tokens use a separate set of query, key, and value projections from the local tokens, the model can learn distinct roles for aggregating global versus local information.

The authors also propose Longformer-Encoder-Decoder (LED), an autoregressive variant that swaps the encoder’s full self-attention for this local-plus-global pattern while keeping the decoder’s full attention over the (short) generated output, extending the approach to long-document generative tasks such as summarization [BPC20].

3.8.2 BigBird

BigBird generalizes this idea by combining local, global, and random attention patterns (see Fig. 5), where each token additionally attends to a small number of randomly sampled tokens [Zah+20]. The authors show that this combination of patterns is a universal approximator of sequence functions and is Turing complete, while still achieving linear complexity, enabling context lengths up to eight times longer than what standard full attention allows on the same hardware [Zah+20].

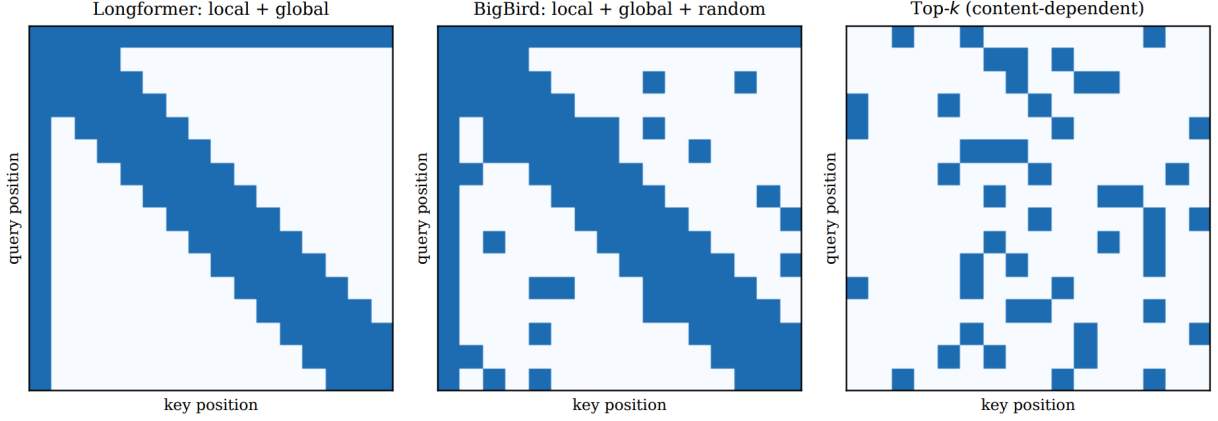


Figure 5: Illustration of different sparse attention mechanisms. (Illustrative, with random scores for top- k and random links.)

This sparsification is motivated by a graph-theoretic view of attention, in which full attention corresponds to a complete graph and the goal is to find a sparser graph that still approximates it well; the authors draw on results from random graph theory, showing that a sparse random graph combined with a small number of global hub nodes has favorable mixing properties similar to a complete graph [Zah+20]. Building on this connection, they prove that BigBird’s attention mechanism can approximate any full attention mechanism to arbitrary precision, i.e. it is a universal approximator of sequence-to-sequence functions, and that it retains the Turing completeness of the standard Transformer, properties that do not automatically follow from sparsity alone [Zah+20].

Empirically, BigBird supports sequences of up to 4096 tokens (versus 512 for BERT-style models on the same hardware), and the authors report improved performance on long-document tasks such as question answering (Natural Questions, HotpotQA) and summarization (arXiv, PubMed), as well as a novel application to genomics sequence data [Zah+20]. BigBird benchmark results are presented in Appendix B.2.

3.9 Linear Attention

Linear attention methods instead reformulate the attention computation itself, replacing the softmax similarity with a kernel feature map $\phi(\cdot)$ so that keys and values are combined before being combined with queries:

$$\text{Attention}(Q, K, V) \approx \phi(Q) (\phi(K)^T V). \quad (10)$$

This avoids ever constructing the $n \times n$ attention matrix explicitly, reducing complexity to $O(n)$, at the cost of only approximating true softmax attention.

3.9.1 Performer

Performer approximates softmax attention using **FAVOR+** (Fast Attention Via positive Orthogonal Random features), a random feature map that provides an unbiased, low-variance estimate of the softmax kernel [Cho+21]. Because the resulting feature maps $\phi(Q)$ and $\phi(K)$ allow attention to be computed as $\phi(Q)(\phi(K)^T V)$, Performer achieves linear time and space complexity while remaining a provably accurate approximation of standard attention, without requiring any sparsity assumptions.

3.9.2 Linformer

Linformer instead observes empirically that the $n \times n$ self-attention matrix is approximately low-rank, and exploits this by projecting the key and value sequence-length dimension down to a much smaller fixed dimension k using learned linear projection matrices [Wan+20]. Attention is then computed between the full queries and the projected, compressed keys and values, reducing both time and memory complexity from $O(n^2)$ to $O(n)$.

3.10 Comparison of Approaches

Table 1 summarizes the main efficiency techniques discussed above, alongside the trade-off each one makes relative to full attention.

Table 1: Comparison of modern attention-efficiency techniques.

Approach	Main Benefit	Trade-off
Full Attention (baseline)	Highest expressiveness and accuracy	$O(n^2)$ compute and memory, large KV cache
GQA	Smaller KV cache, faster decoding	Slightly reduced attention capacity
MQA	Very small KV cache, fastest decoding	Larger quality trade-off than GQA
PagedAttention	Higher GPU utilization, less fragmentation	Additional management / paging overhead
FlashAttention	Same accuracy, lower memory traffic, faster execution	Requires specialized GPU kernels
Quantization	Smaller models, lower bandwidth, faster inference	Possible accuracy degradation
Sparse Attention	Enables much longer contexts, $O(n) \rightarrow O(n \log n)$	May miss important long-range interactions
Linear Attention	Linear $O(n)$ memory and compute scaling	Only approximates softmax attention

The comparison shows that “efficient attention” is not one single technique. GQA and MQA reduce the number of distinct key–value representations. PagedAttention improves memory allocation. FlashAttention reduces data movement while retaining exact attention. Quantization reduces representation size. Sparse and linear methods change the interaction computation itself.

These techniques can also be combined. For example, a serving system may use GQA to reduce cache size, PagedAttention to manage the remaining cache, FlashAttention to reduce attention memory traffic, and quantization to reduce the size of model parameters. The resulting system is adaptive and efficient at multiple levels even when the individual techniques solve different problems.

4 Learned Sparse Attention

Section 3 introduced sparse attention patterns: local, global, and random as a way of reducing the $O(n^2)$ cost of full attention. Those patterns, however, are typically **fixed** in advance by the architecture designer: a given token always attends to its local window, regardless of what that window actually contains. This section discusses a complementary family of techniques in which the sparsity pattern itself is **learned** or computed dynamically from the input, rather than hand-specified.

4.1 Learned and Trainable Dynamic Sparse Attention

Sparse Transformer: Rather than learning sparsity from data, the Sparse Transformer factorizes attention into a small number of fixed sparse patterns whose combination approximates full attention at a fraction of the cost [Chi+19]. A **local** head attends only to the previous l tokens (e.g. $l = 128$), capturing short-range dependencies at cost $O(nl)$. A **strided** head instead attends only to every l -th token before the current position (e.g. positions $i - l, i - 2l, i - 3l, \dots$), capturing long-range, periodic dependencies at the same $O(nl)$ cost. Stacking one local and one strided layer lets information reach any position within two layers, while the total cost drops from $O(n^2)$ to $O(n\sqrt{n})$ when $l \approx \sqrt{n}$ [Chi+19]. The pattern is fixed by the architecture, but which of the two heads (or values within the window) actually matter is still determined dynamically by the learned Q/K content.

Adaptively Sparse Transformers: Adaptively Sparse Transformers go a step further and make the degree of sparsity itself learnable per attention head, by replacing the softmax with the α -entmax transformation [CNM19]:

$$\alpha\text{-entmax}(z) = \operatorname{argmax}_{p \in \Delta^{n-1}} p^T z + H_\alpha(p), \quad (11)$$

where H_α is a generalized entropy and α controls how many entries of the resulting distribution are driven exactly to zero. Unlike the softmax, which always assigns strictly positive weight to every token, entmax can produce attention distributions with exact zeros, giving the model a trainable, per-head notion of how sparse its own attention pattern should be [CNM19].

The α -entmax transformation generalizes the softmax with a single scalar α : at $\alpha = 1$, entmax reduces exactly to the ordinary softmax (dense, all weights strictly positive); at $\alpha = 2$, it becomes *sparsemax*, which produces a distribution with exact zero entries for all but a small subset of tokens [CNM19]. Because α is made a learnable, per-head parameter, training can discover that some heads (e.g. those tracking a single clear antecedent for coreference) benefit from behaving almost like sparsemax, while others (e.g. heads aggregating broad topical context) remain closer to the dense softmax – the sparsity pattern is therefore learned per head rather than fixed by architecture [CNM19].

Reformer: Reformer uses locality-sensitive hashing (LSH) to avoid computing the full QK^T matrix [KKL20]. Each query and key vector is projected through the same random rotation matrix and assigned to a hash bucket based on which rotated region it falls into; vectors that are close in the embedding space are, with high probability, assigned to the same bucket. Attention is then computed *only* between tokens sharing a bucket (with sequences chunked and sorted by bucket to keep computation regular), reducing the effective cost from $O(n^2)$ to roughly $O(n \log n)$ for a sequence of length n [KKL20]. Because the bucket assignment depends on the actual content of Q and K , the resulting sparsity pattern changes from one input sequence to the next.

Routing Transformer: Instead of hashing, the Routing Transformer maintains a small set of k learned cluster centroids (with $k \approx \sqrt{n}$), updated online via an exponential moving average of the queries and keys assigned to each cluster during training [Roy+21]. Every query and key is assigned to its nearest centroid, and attention is computed only between queries and keys sharing a cluster. Because the centroids are updated based on the actual data seen during training, the resulting partition of the sequence adapts to the content being processed, rather than following a fixed positional pattern.

4.2 Dynamic Attention Masks and Content-Aware Sparsity

The Reformer and the Routing Transformer can both be described as applying a **dynamic attention mask**: an additive mask M , computed from the input itself, that sets the attention score of disallowed token pairs to $-\infty$ before the softmax,

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & \text{if } j \text{ is selected for } i \\ -\infty & \text{otherwise.} \end{cases} \quad (12)$$

The key distinction from the causal or local masks discussed earlier is that here the mask M itself depends on the input i.e. it is **content-aware** rather than being a fixed, input-independent pattern determined purely by token position. For Reformer, $M_{ij} = 0$ exactly when i and j fall in the same LSH bucket; for the Routing Transformer, exactly when i and j are assigned to the same cluster centroid.

A further concrete example is **Sparse Sinkhorn Attention**, which learns a differentiable **sorting network** that reorders blocks of the input sequence so that tokens which should attend to each other are moved into the same local block, after which a standard local mask is applied to the reordered sequence [Tay+20]. Because the sorting permutation itself is predicted by a small trained network conditioned on the input, the resulting block structure and therefore the mask M differs from one sequence to the next, rather than being fixed by token position as in a plain sliding-window mask.

4.3 Top-k Attention and Dynamic Sparsity

A particularly simple and widely used form of content-aware sparsity is **top-k attention**: for each query, only the k keys with the highest raw attention score are retained, and the softmax is computed only over this reduced set,

$$\mathcal{T}_i = \text{topk}_j(q_i k_j^T), \quad A_i = \text{softmax}_{j \in \mathcal{T}_i}\left(\frac{q_i k_j^T}{\sqrt{d_k}}\right)v_j. \quad (13)$$

The **Explicit Sparse Transformer** applies exactly this rule at training time: for every query, only the top- k scoring keys are kept (with k a small constant such as 8 or

16, regardless of sequence length n), and the remaining scores are masked to $-\infty$ before renormalizing, which the authors show removes noisy, low-relevance attention weights and improves accuracy on top of reducing cost [Zha+19].

Because the selected set $\mathcal{T}_i$ depends on the query and key content, this constitutes a dynamic sparsity pattern computed on the fly, without requiring any additional learned parameters beyond the standard query, key projections. The trade-off is that the top- k operation is non-differentiable, which either requires approximating it with a soft relaxation during training or accepting a mismatch between training and inference sparsity.

A second, deployment-oriented example is the **H₂O** (“**Heavy-Hitter Oracle**”) cache-eviction policy for autoregressive decoding: rather than applying top- k within a single forward pass, it tracks a running estimate of cumulative attention received by each cached key across decoding steps and dynamically retains only the top- k “heavy-hitter” tokens in the KV cache, evicting the rest to keep long-context inference tractable [Zha+23].

5 Adaptive Attention Span

A closely related idea is to make not just **which** tokens are attended to dynamic, but **how far back** a given attention head is allowed to look i.e. its **span** which is an adaptive, learned quantity rather than a fixed architectural hyperparameter.

5.1 Learnable Attention Span

In the standard Transformer, every attention head has the same maximum context length, fixed by the sequence length or, in sliding-window variants, by a manually chosen window size. The Adaptive Attention Span model instead equips each attention head with its own **learnable span parameter** z , which is used to construct a soft masking function $m_z(x)$ that smoothly decays the attention weight for tokens further than z positions away [Suk+19]:

$$m_z(x) = \text{clip}\left(\frac{1}{R}(R + z - x), 0, 1\right), \quad (14)$$

where x is the distance between the query and key positions and R controls the softness of the cutoff. A penalty proportional to z is added to the training loss, so the model is directly incentivized to learn the **smallest** span that does not hurt performance for each head [Suk+19].

5.2 Input-Dependent Context Length

A related but distinct idea is to let the effective context available to the model as a whole grow dynamically with the input, rather than fixing it per head. Transformer-XL introduces segment-level recurrence: hidden states computed for a previous segment of text are cached and reused as an extended context for the next segment, effectively letting information flow across segment boundaries without recomputing attention over the entire history [Dai+19]. Combined with relative positional encodings, this allows the model’s effective receptive field to grow far beyond the length of any single training segment, adapting to how much prior context is actually available at inference time [Dai+19].

5.3 Dynamic Masking and Adaptive Context Windows

Taken together, learnable span and segment-level recurrence can both be described as forms of **dynamic masking**: instead of a binary local-window mask fixed at model design time, the effective window, whether continuous (as in adaptive span) or discrete and recurrent (as in Transformer-XL) is adjusted per head or per segment based on what is learned to be useful during training. The practical benefit is the same as for dynamic sparsity in Section 4: most of the $O(n^2)$ cost of full attention is avoided, but here specifically by adaptively bounding how far attention needs to reach, rather than by selecting an arbitrary subset of distant tokens.

6 Dynamic Token Selection, Pruning, and Merging

The techniques discussed so far change which connections within attention are computed. A complementary strategy is to **reduce the number of tokens** that participate in computation at all, by dynamically identifying and removing or merging tokens that contribute little to the final output as they pass through the network.

6.1 Adaptive and Importance-Based Token Selection

Not all tokens are equally important to a model’s final prediction: in classification tasks, for instance, many tokens contribute little beyond what is already captured by a handful of salient words. PoWER-BERT exploits this by learning, for each layer, to retain only the subset of tokens with the highest attention-derived **significance score**, computed by summing the attention each token receives from the [CLS] token across all heads [Goy+20]. Because this significance score is recomputed at every layer from the current token representations, the set of retained tokens is genuinely **adaptive** to the input, rather than fixed by position.

6.2 Token Pruning and Progressive / Dynamic Eviction

Building on this idea, Learned Token Pruning learns, per layer, a threshold below which tokens are discarded based on their attention scores, allowing the model to prune a different number of tokens for different inputs depending on how much redundancy is actually present [Kim+22]. This is naturally **progressive**: because pruning is applied at multiple layers in sequence, the number of active tokens shrinks gradually as the sequence moves deeper into the network, rather than being cut all at once.

6.3 Token Merging and Candidate Token Selection

Rather than discarding tokens outright, an alternative is to **merge** redundant tokens into a smaller set of representative tokens, preserving more of their information. Token Merging (ToMe) identifies pairs of similar tokens at each layer using key vectors as a similarity signal and merges a fixed number of the most similar pairs at every layer via averaging, gradually reducing the number of tokens as depth increases without requiring any retraining [Bol+23].

In vision Transformers, DynamicViT instead learns a lightweight prediction module that scores each token’s importance and progressively selects a shrinking set of **candidate tokens** to keep, using a differentiable, Gumbel-softmax-based relaxation of the discrete selection decision so that the whole pipeline remains trainable end-to-end [Rao+21].

7 Conclusion

Attention was originally introduced to solve the information bottleneck and vanishing-gradient issues of recurrent sequence-to-sequence models. The Transformer architecture generalized this idea into a complete replacement for recurrence, using scaled dot-product self-attention and multi-head attention as its primary mechanism for modeling dependencies between tokens, whether within the encoder, within the masked decoder, or across the encoder-decoder boundary.

As Transformer-based LLMs have scaled to ever-larger parameter counts and context lengths, the central challenge has shifted. Raw expressiveness is no longer the primary bottleneck; instead, memory bandwidth and the size of the key-value cache required for autoregressive decoding have become the dominant constraints, a phenomenon summarized by the notion of the memory wall. This has driven a wave of architectural and systems-level innovations like GQA, MQA, PagedAttention, FlashAttention, quantization, sparse and linear attention all aimed at making attention not just powerful, but efficient enough to run at the scale demanded by modern applications.

Beyond improving the efficiency of attention, recent Transformer research has increasingly focused on making computation more dynamic and adaptive. Dynamic attention mechanisms allow the model to make input-dependent decisions about which information is relevant, while learned sparse attention further reduces computation by selectively activating only a subset of token interactions. Adaptive attention span extends this idea by allowing different attention heads to learn how much contextual information they actually require, rather than applying a fixed context range uniformly. At the token level, dynamic token selection, pruning, and merging reduce computational cost by identifying redundant or low-importance representations and progressively removing or combining them.

Taken together, the methods discussed in this report show that the future development of Transformer-based systems involves not only increasing model capacity, but also making computation more selective, dynamic, adaptive, memory-efficient, and interpretable. Attention has therefore evolved from a mechanism for selectively focusing on relevant information into a broader framework for dynamically allocating computation across modern neural networks.

References

- [Ain+23] Joshua Ainslie et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. arXiv: 2305.13245 [cs.CL]. URL: <https://arxiv.org/abs/2305.13245>.
- [Ala18] Jay Alammar. *The Illustrated Transformer*. Blog post. June 2018. URL: <https://jalammar.github.io/illustrated-transformer/>.
- [AZ20] Samira Abnar and Willem Zuidema. “Quantifying Attention Flow in Transformers”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020. arXiv: 2005.00928 [cs.LG]. URL: <https://arxiv.org/abs/2005.00928>.
- [BCB15] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- [BM] Lisa Beinborn and Jonas Mayer Martins. *Language as Data, Interpretability and Bias of Machine Learning Models*. Lecture notes, Institute of Computer Science, University of Göttingen.
- [Bol+23] Daniel Bolya et al. “Token Merging: Your ViT but Faster”. In: *arXiv preprint arXiv:2210.09461* (2023). arXiv: 2210.09461 [cs.CV]. URL: <https://arxiv.org/abs/2210.09461>.
- [BPC20] Iz Beltagy, Matthew E. Peters, and Arman Cohan. “Longformer: The Long-Document Transformer”. In: 2020. arXiv: 2004.05150 [cs.CL]. URL: <https://arxiv.org/abs/2004.05150>.
- [CGW21] Hila Chefer, Shir Gur, and Lior Wolf. “Transformer Interpretability Beyond Attention Visualization”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021. arXiv: 2012.09838 [cs.CV]. URL: <https://arxiv.org/abs/2012.09838>.
- [Chi+19] Rewon Child et al. “Generating Long Sequences with Sparse Transformers”. In: *arXiv preprint arXiv:1904.10509* (2019). arXiv: 1904.10509 [cs.LG]. URL: <https://arxiv.org/abs/1904.10509>.
- [Cho+21] Krzysztof Choromanski et al. “Rethinking Attention with Performers”. In: *Proceedings of the 9th International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2009.14794 [cs.LG]. URL: <https://arxiv.org/abs/2009.14794>.
- [CNM19] Gonalo M. Correia, Vlad Niculae, and Andr  F. T. Martins. “Adaptively Sparse Transformers”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019. arXiv: 1909.00015 [cs.CL]. URL: <https://arxiv.org/abs/1909.00015>.

- [Dai+19] Zihang Dai et al. “Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2019. arXiv: 1901.02860 [cs.LG]. URL: <https://arxiv.org/abs/1901.02860>.
- [Dao+22] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022. arXiv: 2205.14135 [cs.LG]. URL: <https://arxiv.org/abs/2205.14135>.
- [Eck] Alexander Ecker. *Machine Learning - Self Attention & Transformers*. Lecture notes, Institute of Computer Science, University of Göttingen.
- [Goy+20] Saurabh Goyal et al. “PoWER-BERT: Accelerating BERT Inference via Progressive Word-vector Elimination”. In: *arXiv preprint arXiv:2001.08950* (2020). arXiv: 2001.08950 [cs.CL]. URL: <https://arxiv.org/abs/2001.08950>.
- [Kim+22] Sehoon Kim et al. “Learned Token Pruning for Transformers”. In: *arXiv preprint arXiv:2107.00910* (2022). arXiv: 2107.00910 [cs.LG]. URL: <https://arxiv.org/abs/2107.00910>.
- [KKL20] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. “Reformer: The Efficient Transformer”. In: *Proceedings of the 8th International Conference on Learning Representations (ICLR)*. 2020. arXiv: 2001.04451 [cs.LG]. URL: <https://arxiv.org/abs/2001.04451>.
- [Kwo+23] Woosuk Kwon et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. 2023. arXiv: 2309.06180 [cs.LG]. URL: <https://arxiv.org/abs/2309.06180>.
- [LPM15] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. “Effective Approaches to Attention-based Neural Machine Translation”. In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2015. arXiv: 1508.04025 [cs.CL]. URL: <https://arxiv.org/abs/1508.04025>.
- [Ola15] Christopher Olah. *Understanding LSTM Networks*. Blog post, colah’s blog. Aug. 2015. URL: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- [Pop+23] Reiner Pope et al. “Efficiently Scaling Transformer Inference”. In: *arXiv preprint arXiv:2211.05102* (2023). arXiv: 2211.05102 [cs.LG]. URL: <https://arxiv.org/abs/2211.05102>.
- [Rao+21] Yongming Rao et al. “DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification”. In: *arXiv preprint arXiv:2106.02034* (2021). arXiv: 2106.02034 [cs.CV]. URL: <https://arxiv.org/abs/2106.02034>.
- [Roy+21] Aurko Roy et al. “Efficient Content-Based Sparse Attention with Routing Transformers”. In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 53–68. arXiv: 2003.05997 [cs.LG]. URL: <https://arxiv.org/abs/2003.05997>.
- [Su+21] Jianlin Su et al. “RoFormer: Enhanced Transformer with Rotary Position Embedding”. In: 2021. arXiv: 2104.09864 [cs.CL]. URL: <https://arxiv.org/abs/2104.09864>.

- [Suk+19] Sainbayar Sukhbaatar et al. “Adaptive Attention Span in Transformers”. In: *arXiv preprint arXiv:1905.07799* (2019). arXiv: 1905.07799 [cs.LG]. URL: <https://arxiv.org/abs/1905.07799>.
- [Tay+20] Yi Tay et al. “Sparse Sinkhorn Attention”. In: *arXiv preprint arXiv:2002.11296* (2020). arXiv: 2002.11296 [cs.LG]. URL: <https://arxiv.org/abs/2002.11296>.
- [Tay+21] Yi Tay et al. “Long Range Arena: A Benchmark for Efficient Transformers”. In: *Proceedings of the 9th International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2011.04006 [cs.LG]. URL: <https://arxiv.org/abs/2011.04006>.
- [Uni23] University of Washington. *CSE 543: Deep Learning (Autumn 2023)*. Instructor: Simon S. Du. 2023. URL: <https://courses.cs.washington.edu/courses/cse543/23au/> (visited on 06/09/2026).
- [Vas+17] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [Wan+20] Sinong Wang et al. “Linformer: Self-Attention with Linear Complexity”. In: *arXiv preprint arXiv:2006.04768* (2020). arXiv: 2006.04768 [cs.LG]. URL: <https://arxiv.org/abs/2006.04768>.
- [Wel25] Welch Labs. *How DeepSeek Rewrote the Transformer [MLA]*. YouTube video. Mar. 2025. URL: https://www.youtube.com/watch?v=OVL AoVGf_74 (visited on 06/09/2026).
- [WP19] Sarah Wiegrefe and Yuval Pinter. “Attention is not not Explanation”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 11–20. URL: <https://aclanthology.org/D19-1002/>.
- [Xu+15] Kelvin Xu et al. “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention”. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Vol. 37. PMLR. 2015, pp. 2048–2057. arXiv: 1502.03044 [cs.LG]. URL: <https://arxiv.org/abs/1502.03044>.
- [Xu+23] Li Xu et al. “Attribution rollout: A new way to interpret visual transformer”. In: *Journal of Ambient Intelligence and Humanized Computing* 14 (2023), pp. 163–173. DOI: 10.1007/s12652-022-04354-2. URL: <https://doi.org/10.1007/s12652-022-04354-2>.
- [Zah+20] Manzil Zaheer et al. “Big Bird: Transformers for Longer Sequences”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020. arXiv: 2007.14062 [cs.LG]. URL: <https://arxiv.org/abs/2007.14062>.
- [Zha+19] Guangxiang Zhao et al. “Explicit Sparse Transformer: Concentrated Attention Through Explicit Selection”. In: *arXiv preprint arXiv:1912.11637* (2019). arXiv: 1912.11637 [cs.CL]. URL: <https://arxiv.org/abs/1912.11637>.
- [Zha+23] Zhenyu Zhang et al. “H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models”. In: *arXiv preprint arXiv:2306.14048* (2023). arXiv: 2306.14048 [cs.LG]. URL: <https://arxiv.org/abs/2306.14048>.

A Transformer Architecture and Model Representation

A.1 Attention in the Encoder and Decoder

Encoder self-attention: In encoder self-attention, queries, keys, and values all originate from the encoder representation. This lets every input token attend to every other input token, including tokens that appear later in the sequence, since the encoder has access to the full input at once.

Masked decoder self-attention: The decoder must generate tokens **autoregressively**, one at a time, and may therefore not be allowed to look ahead at tokens it has not yet generated. In decoder self-attention, a **causal mask** restricts the attention matrix so that a position cannot attend to future positions. This preserves the autoregressive property.

Encoder-decoder cross-attention. After masked self-attention, the decoder performs a second attention step in which the queries come from the decoder’s own hidden states, while the keys and values come from the encoder’s output representations. This cross-attention step allows each token being generated to attend to the relevant parts of the source sequence. Thus, the decoder can dynamically select relevant parts of the source sequence while producing each output token [Vas+17].

A.2 Positional Information: Rotary Positional Embeddings

Because self-attention treats its input as an unordered set of tokens, it is by construction **permutation-invariant** and has no inherent notion of token order [Su+21]. Positional information must therefore be injected explicitly. RoPE does this by **rotating the query and key vectors** in vector space by an angle that depends on the token’s position p , before the attention scores are computed [Su+21]:

$$q'_p = \theta_p q_p, \quad k'_p = \theta_p k_p, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q'K'^T}{\sqrt{d_k}}\right) V. \quad (15)$$

Because rotation preserves vector norms and the relative angle between two rotated vectors depends only on the difference between their positions, RoPE naturally encodes relative position information directly inside the dot-product attention score, without requiring separate additive positional embeddings [Su+21]. This property has made RoPE the de facto standard for positional encoding in most modern LLMs.

A.3 Quantization

Quantization refers to the process of representing model parameters and, in some cases, intermediate activations using lower numerical precision than the original representation. Instead of storing every parameter using high-precision floating-point values such as FP32 or FP16, quantization can represent these values using formats such as INT8 or INT4. The primary motivation is to reduce the memory footprint and computational cost of large language models while retaining as much of their original performance as possible.

For a 70-billion-parameter model, this corresponds to roughly 280 GB at FP32, 140 GB at FP16, 70 GB at INT8, and 35 GB at INT4 (see Fig. 6).

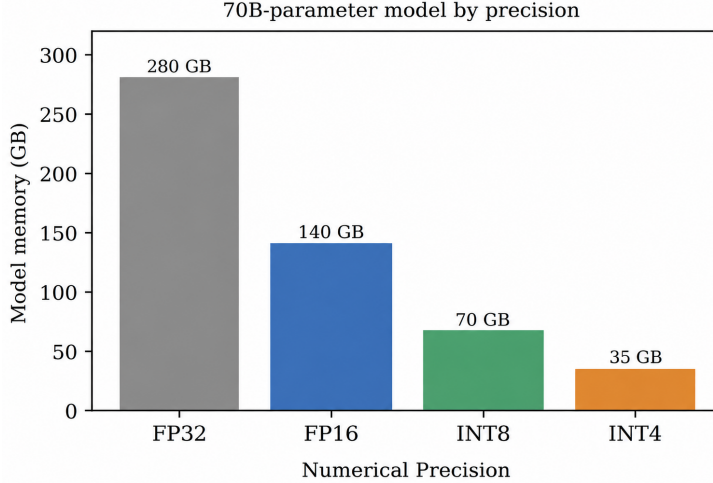


Figure 6: Memory of a 70B-parameter model at different precisions.

B Sparse Attention Benchmark Comparisons

A lower complexity class says nothing about whether the omitted interactions were actually irrelevant. The Long Range Arena (LRA) benchmark was introduced specifically to test this, evaluating efficient attention variants on five long-sequence classification tasks (ListOps, byte-level text classification, document retrieval, image classification, and Pathfinder) using an identical architecture and training budget for every model, so that accuracy differences can be attributed to the attention pattern alone rather than to confounding factors [Tay+21]. Table 2 reports the average accuracy for full attention alongside the two sparse variants.

Table 2: Average accuracy (%) on the Long Range Arena benchmark [Tay+21]. All models share the same backbone and training setup; only the attention pattern differs.

Model	Complexity	Avg. Accuracy (%)	vs. Full Attention
Full Attention (baseline)	$O(n^2)$	54.39	—
BigBird	$O(n)$	55.01	+0.62
Longformer	$O(n)$	53.46	−0.93

BigBird’s combination of local, global, and random attention slightly exceeds full attention on average (55.0% vs. 54.4%), while Longformer’s purely local-plus-global pattern falls slightly short (53.5%), and every model in the benchmark including full attention fails outright on the hardest task, Path-X, which requires reasoning about pixel-level connectivity across sequences of 16K tokens [Tay+21]. This shows that a well-chosen sparsity pattern may not cost accuracy on tasks the pattern is suited to, but also that no amount of clever sparsification currently substitutes for the underlying difficulty of certain long-range reasoning tasks.

B.1 Longformer

On character-level language modeling, Longformer’s small and large configurations achieve state-of-the-art bits-per-character on both `text8` (1.10 bpc) and `enwik8` (1.00 bpc for the small model), matching or beating prior sparse and full-attention Transformers of comparable size while processing sequences of length 4,096 to 23,040 tokens [BPC20]. Longformer was also pretrained and fine-tuned end-to-end and compared directly against a RoBERTa baseline that truncates long documents into 512-token chunks. As Table 3 shows, Longformer-base matches or exceeds RoBERTa-base on every task despite using linear-complexity attention, with the largest gains appearing precisely on the tasks that most require long-range context, such as WikiHop [BPC20].

Table 3: Longformer-base vs. RoBERTa-base on long-document QA (development sets) [BPC20].

Task (metric)	RoBERTa-base	Longformer-base	Δ
WikiHop (accuracy)	72.4	75.0	+2.6
TriviaQA (F1)	74.3	75.2	+0.9
HotpotQA (joint F1)	63.5	64.4	+0.9

B.2 BigBird

BigBird’s own evaluation shows the benefit of adding random attention on top of Longformer’s local-plus-global pattern. Using the same base-model scale and the same QA datasets as Longformer, both ITC (internal transformer construction) and ETC (extended transformer construction) variants of BigBird outperform both the RoBERTa baseline and Longformer on every task, and by a wider margin than Longformer’s own gain over RoBERTa (Table 4) [Zah+20]. The largest gap appears on TriviaQA, where BigBird-ITC reaches 79.5 F1 against Longformer’s 75.2 and RoBERTa’s 74.3 [Zah+20]. This is consistent with the LRA results above: adding a small amount of *random* long-range connectivity to the local-plus-global pattern recovers some of the modeling capacity that pure local-plus-global attention gives up, without paying the full $O(n^2)$ cost.

Table 4: QA development-set results, base-size models: RoBERTa vs. Longformer vs. BigBird [Zah+20].

Task (metric)	RoBERTa	Longformer	BigBird-ITC	BigBird-ETC
WikiHop (accuracy)	72.4	75.0	75.9	75.9
TriviaQA (F1)	74.3	75.2	79.5	78.7
HotpotQA (joint F1)	63.5	64.4	67.7	67.8

C Dynamic Attention Mechanisms

Standard Transformer attention is already dynamic at the level of its weights. Given Q , K , and V , the attention operation is

$$A(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (16)$$

Because queries and keys are computed from the current representations, the attention distribution changes with the input. Dynamic attention extends this basic property by making the **amount, location, or structure** of computation input-dependent.

C.1 Query-Dependent Attention

Unlike a convolution or a fixed-size pooling operation, whose weights are learned once during training and then applied identically regardless of the input, attention scores are computed **at inference time** as a function of the current input [BCB15]. Concretely, the scaled dot-product formulation produces a different attention distribution for every query vector q_i , since the scores $q_i k_j^\top$ depend directly on the content of both the query and the key. This is what is meant by **query-dependent** attention: the same key token k_j may receive high weight from one query and negligible weight from another, purely because of how well their representations align [LPM15]. This stands in contrast to earlier, non-attentive sequence models, where the influence of a given input position on the output was essentially fixed by its position rather than by its content.

For example, consider the Winograd-style sentence pair “The animal didn’t cross the street because *it* was too tired” versus “...because *it* was too wide.” The key vectors for *animal* and *street* are fixed once the sentence has been encoded, but the query vector produced for the pronoun *it* differs between the two sentences, since it is influenced by the adjective that follows (*tired* vs. *wide*). As a result, the same key for *animal* receives high attention weight when the query is conditioned on *tired*, while the key for *street* receives high weight when the query is conditioned on *wide*. The identical token pair (*it*, *animal*) is scored completely differently depending on the query alone [Uni23; Eck].

C.2 Context-Dependent and Content-Aware Attention

Attention is also context-dependent because the representations used to construct queries and keys are themselves contextualized. Content-aware attention emphasizes that relevance is determined by the representations of the tokens rather than by position alone. A distant token can receive high attention when its content is relevant, while a nearby token can receive little attention when it contributes little to the current computation.

This distinction becomes important when attention is made sparse. A sparse system needs a criterion for deciding which interactions should remain active. Content similarity, learned relevance scores, attention scores, or auxiliary routing networks can provide such signals.

For example, a polysemous word such as *bank* illustrates content-awareness directly. In the sentence “I sat by the river *bank*”, the self-attention layer produces a key/value pair for *bank* that is influenced by its own embedding together with the embeddings of nearby words after the first layer’s mixing; the word *river* receives comparatively high

attention weight from *bank*’s query because their contextual representations are close in the shared embedding space, and this shapes the contextualized representation of *bank* toward the “riverbank” sense. In the sentence “I opened a *bank* account”, the same token *bank* instead attends strongly to *account*, steering its representation toward the financial-institution sense. The identical token thus receives a different attention pattern purely because of the surrounding *content*, not its position in the sentence [BM].

C.3 Dynamic and Importance-Based Attention Allocation

A useful way to summarize the mechanisms above is that attention performs **dynamic allocation** of a fixed computational “budget” i.e. the probability mass of the softmax distribution across the tokens available to it. Rather than treating all input positions as equally important, the model learns to concentrate this budget on the subset of tokens that are most relevant, or most important, for producing a given output, and to allocate correspondingly little weight to irrelevant tokens [Uni23]. This notion of importance-based allocation motivates all of the more specialized mechanisms: if a model can already learn which tokens matter most for a given query, it is natural to ask whether it can also **learn to skip computing attention** to the tokens it would assign negligible weight to in the first place. This question motivates dynamic and learned sparse attention.

For example, in the French-to-English translation (“il a m’entarté” → “he hit me with a pie”), when the decoder generates the word *hit*, the attention distribution over the four source tokens is highly peaked: roughly most of the probability mass concentrates on the source token *entarté* (the verb), while the remaining source tokens each receive only a few percent [Uni23]. The model has learned to allocate almost its entire attention budget to the one source token that is actually relevant for producing the current output word, rather than distributing weight uniformly across all four source positions. This asymmetric, input-dependent allocation is what is meant by *importance-based* attention allocation.

D Attention Interpretability

Because attention weights are, by construction, a set of normalized scores describing how much one token relies on another, it is tempting to interpret them directly as an explanation of model behavior. Early work on visual attention illustrated this appeal vividly: in image captioning models, the region of an image attended to when generating a given word aligns strikingly well with the object that word refers to [Xu+15]. However, more careful analysis has shown that raw attention weights in a single layer can be a misleading proxy for importance, since information from a given token is **mixed and transformed across many layers** before it influences the final output [CGW21]. This motivates methods that aggregate attention across the full depth of the network.

D.1 Attention as a Graph

One useful way to reason about how information propagates through attention layers is to model the network as a **DAG** [AZ20]. In this view, input tokens and intermediate hidden representations at each layer are the **nodes** of the graph, and the attention weight from a node in one layer to a node in the previous layer is the **weight of the edge** connecting

them [AZ20]. A **path** from a node v at position k in layer l_i to a node u at position m in layer l_j is then simply a sequence of such edges connecting the two [AZ20].

D.2 Attention Rollout

Attention rollout estimates how much information at an early node v is propagated to a later node u by multiplying the edge weights along a given path, and then summing this quantity over all possible paths between v and u [AZ20]. Intuitively, this recursively “rolls out” the raw per-layer attention distributions into a single, cumulative attention map with respect to the original input tokens, rather than only the immediately preceding layer. This produces saliency-style visualizations that highlight which input regions most influenced a given output, and has since been extended from language models to Vision Transformers, where it can be used to produce heatmaps over image patches that closely track the object being classified [Xu+23].

D.3 Attention Flow

Attention flow adapts the same graph formulation but reasons about it using ideas from maximum-flow network theory instead of simple path summation [AZ20]. Each edge is treated as having a fixed **capacity**, given by its attention weight, and the flow through any single edge is not allowed to exceed that capacity. The weight of an entire path is then taken to be the **minimum** edge weight along that path (the bottleneck of the path), rather than the product of all its edge weights [AZ20]. Because many paths between two nodes can overlap and share edges, computing the true maximum flow between a source and a sink node is more involved than simple rollout, but tends to produce attribution maps that are more robust to the fact that attention weights are not perfectly comparable across independent paths [AZ20].